

PROJECT REPORT

Text-Based Adventure Game in C++

"The Lost Kingdom of Eldoria"

Internship Task Submission Organization: Pinnacle Labs **Task:** Develop a Text-Based Adventure Game in C++ **Developer:** Priyam Prakash **Date:** May 2026

TABLE OF CONTENTS

1. Introduction
2. Objectives
3. Features
4. System Requirements
5. Project Architecture
6. Module Descriptions
7. Game Design — Story & Branching Narrative
8. Object-Oriented Programming Concepts Used
9. Data Structures Used
10. Key Algorithms & Logic
11. How to Compile and Run
12. Sample Output / Gameplay Screenshots
13. Testing & Validation
14. Challenges Faced & Solutions
15. Future Enhancements
16. Conclusion
17. Source Code File Listing

1. INTRODUCTION

"The Lost Kingdom of Eldoria" is an interactive text-based adventure game developed in C++ as part of the Pinnacle Labs internship program. The game presents players with a captivating fantasy story

where they navigate through a branching narrative, make critical decisions, engage in turn-based combat, and experience multiple endings based on their choices.

Text-based adventure games are a foundational genre in game design that emphasize storytelling, player agency, and decision-making. This project demonstrates proficiency in C++ programming, object-oriented design, data structure management, and software engineering principles.

2. OBJECTIVES

The primary objectives of this project are:

- **Develop an interactive game** with a captivating story and immersive gameplay experience.
- **Implement branching narratives** where user choices directly affect the story outcome.
- **Apply Object-Oriented Programming (OOP)** principles including encapsulation, abstraction, and modular design.
- **Demonstrate proficiency in C++** including STL containers, file I/O, enums, structs, and classes.
- **Create variable outcomes** — multiple endings determined by player decisions throughout the game.
- **Implement a combat system** with strategic depth (attack, defend, use items, flee).
- **Build a save/load system** for persistent game state across sessions.

3. FEATURES

3.1 Core Features

Feature	Description
Branching Storyline	25+ story nodes with interconnected paths
Multiple Endings	5 unique endings (Hero, Dark, Guardian, Ascended, Death)
Character Classes	3 playable classes — Warrior, Mage, Rogue
Turn-Based Combat	Strategic combat with attack, defend, item use, and flee
Inventory System	15-slot inventory with consumables, equipment, and key items
Karma System	Moral choice tracker that influences story outcomes
Save/Load System	File-based game persistence with auto-save at checkpoints
Leveling System	Experience-based progression with stat upgrades

3.2 Visual & UX Features

Feature	Description
Colored Text	Windows Console API color-coded text for different elements
Typewriter Effect	Animated text display for narrative immersion
Health Bars	Visual HP bars for player and enemies during combat
ASCII Art	Styled title banner and Game Over screen
Boxed Menus	Formatted menu displays with numbered choices
Input Validation	Robust input handling preventing crashes from invalid input

4. SYSTEM REQUIREMENTS

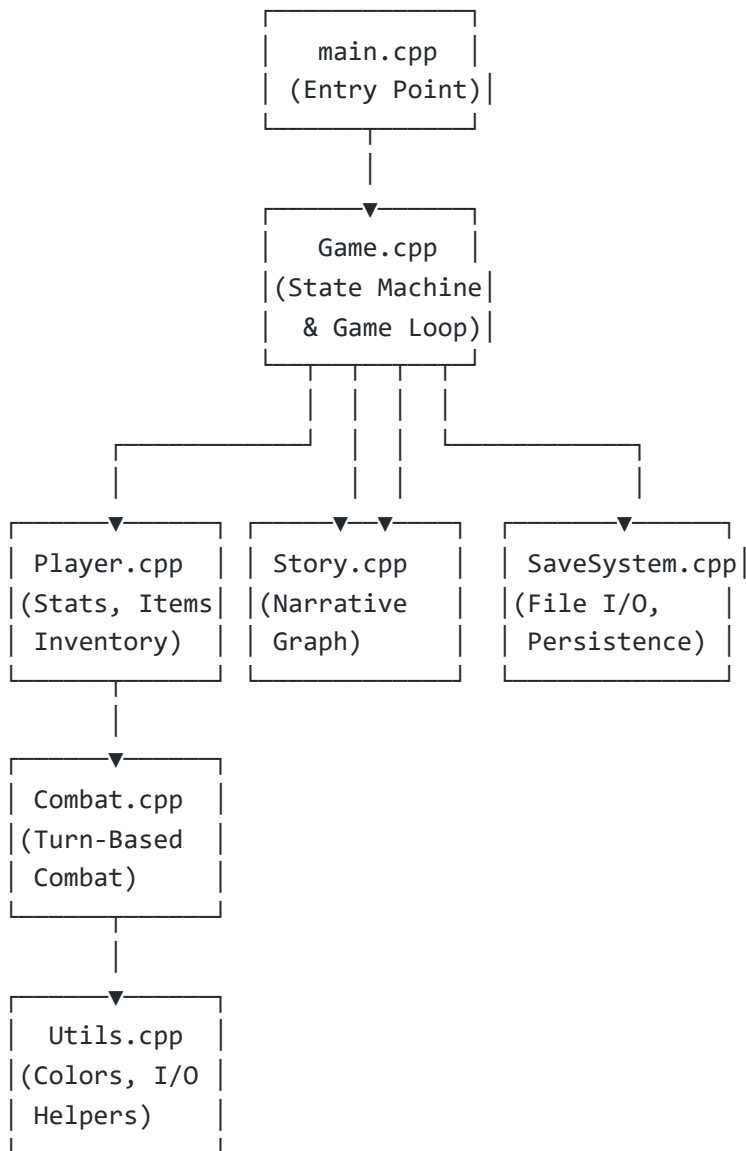
- **Operating System:** Windows 10/11
- **Compiler:** g++ (MinGW) with C++17 support
- **RAM:** 512 MB (minimal)
- **Disk Space:** < 1 MB
- **Console:** Windows Command Prompt or PowerShell

5. PROJECT ARCHITECTURE

The project follows a modular architecture with clear separation of concerns:

```
text based adventure game/
|
├─ main.cpp           - Entry point, console setup
├─ Game.h / Game.cpp  - Game engine (state machine, main loop)
├─ Player.h / Player.cpp - Player stats, inventory, serialization
├─ Story.h / Story.cpp - Narrative graph (story nodes & choices)
├─ Combat.h / Combat.cpp - Turn-based combat system
├─ Utils.h / Utils.cpp - Utility functions (colors, I/O, menus)
├─ SaveSystem.h / SaveSystem.cpp - File-based save/load
├─ README.md          - Project documentation
└─ PROJECT_REPORT.md  - This report
```

Architectural Diagram



6. MODULE DESCRIPTIONS

6.1 main.cpp — Entry Point

- Sets up Windows console for UTF-8 character support
- Instantiates the `Game` object and launches `game.run()`
- Minimal and clean entry point following best practices

6.2 Game.h / Game.cpp — Game Engine

- Implements a **finite state machine** with states: `MAIN_MENU` , `CHARACTER_CREATION` , `PLAYING` , `INVENTORY_MENU` , `GAME_OVER` , `VICTORY`
- Manages the main game loop via `run()` method
- Handles story node processing, choice routing, combat triggering, and ending determination

- Includes logic for the secret "Ascended" ending (requires high karma + specific items)

6.3 Player.h / Player.cpp — Player Management

- **Item struct** — name, description, heal amount, attack/defense boosts, consumable/key flags
- **CharacterClass enum** — WARRIOR (tanky), MAGE (high damage, crits), ROGUE (balanced, bonus gold, high flee chance)
- **Player class** — manages health, attack, defense, gold, level, experience, karma, and inventory
- `serialize()` / `deserialize()` methods for save/load support
- Visual stat display with colored health bar

6.4 Story.h / Story.cpp — Narrative System

- **Choice struct** — text, destination node, karma change, required item
- **StoryNode struct** — id, title, narrative text, choices, combat flag, item rewards, healing, ending flag
- **Story class** — builds the entire narrative graph in `buildStory()` with 25+ interconnected nodes
- Supports conditional choices (requiring specific items to access certain paths)

6.5 Combat.h / Combat.cpp — Combat System

- **Enemy struct** — name, health, attack, defense, rewards, taunt message
- **Factory functions** — `createForestWolf()`, `createGoblinScout()`, `createSkeletonWarrior()`, `createDarkMage()`, `createBanditLeader()`, `createShadowDragon()`
- **Combat loop** — player and enemy take alternating turns
- Combat options: Attack (with Mage critical hits), Defend (halves damage), Use Item, Flee (Rogue bonus)
- Visual HP bars for both player and enemy during combat

6.6 Utils.h / Utils.cpp — Utility Functions

- Console color management using Windows API (`SetConsoleTextAttribute`)
- Typewriter text effect for immersive narrative display
- Screen clearing, separator lines, boxed titles
- Robust input validation (`getValidInput` , `getStringInput` , `getYesNo`)
- Menu display system with numbered options
- Random number generation using C++11 `<random>` library

6.7 SaveSystem.h / SaveSystem.cpp — Persistence

- Saves player state and current story node to `savegame.dat`
- Uses pipe-delimited serialization format for player data

- `saveFileExists()` check enables conditional "Continue" menu option
- Auto-save at key story checkpoints (nodes 13, 20, 25)
- Save file deleted upon reaching any ending

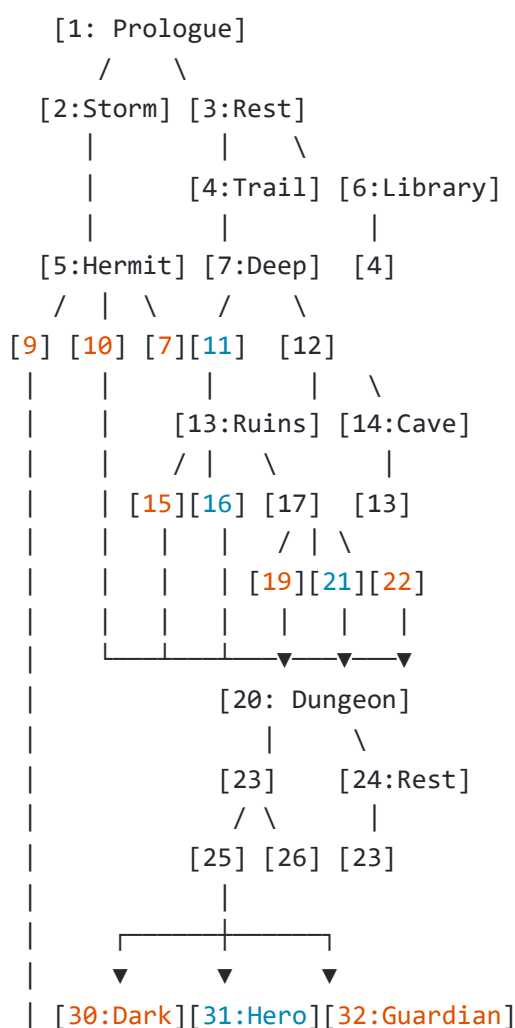
7. GAME DESIGN — STORY & BRANCHING NARRATIVE

7.1 Story Overview

The player discovers an ancient map leading to the Lost Kingdom of Eldoria, a once-glorious kingdom now sealed away by powerful magic. The journey takes the player through:

1. **The Abandoned Library** — Discovery of the ancient map
2. **The Enchanted Forest** — Magical creatures and moral choices
3. **The Ancient Ruins** — Exploration and artifact collection
4. **The Dark Dungeon** — Final challenges and the boss battle
5. **The Throne Room** — Confrontation with the Shadow Dragon

7.2 Branching Path Diagram





* Secret ending: requires karma ≥ 10 + Crystal of Light + Light Amulet

7.3 The Five Endings

Ending	Node	Condition	Description
The Dark Sovereign	30	Claim throne for yourself	Player becomes the new dark ruler
The Restorer	31	Restore the kingdom	Player rebuilds Eldoria as a hero
The Guardian	32	Seal kingdom forever	Player becomes a wandering protector
The Ascended	33	High karma + both artifacts	Secret true ending — player ascends to godhood
Fallen Adventurer	34	Die in combat	Player falls and becomes a cautionary tale

8. OBJECT-ORIENTED PROGRAMMING CONCEPTS USED

8.1 Encapsulation

- All class member variables are **private** with public getter/setter methods
- Example: `Player` class hides `health`, `attack`, `inventory` behind accessor methods

8.2 Abstraction

- Complex operations are abstracted behind simple interfaces
- Example: `Game::run()` abstracts the entire game loop; `runCombat()` abstracts the combat system

8.3 Modularity

- Each module has a single responsibility (Single Responsibility Principle)
- Header (.h) and implementation (.cpp) file separation
- Clean `#ifndef` include guards prevent double-inclusion

8.4 Data Structures as Objects

- `Item`, `Enemy`, `Choice`, `StoryNode` — structured data types encapsulating related fields

- `Player` — full class with behavior (methods) and state (member variables)

8.5 Enumerations

- `CharacterClass` — WARRIOR, MAGE, ROGUE
- `GameState` — MAIN_MENU, CHARACTER_CREATION, PLAYING, etc.
- `Color` — console color codes

9. DATA STRUCTURES USED

Data Structure	Usage	Location
<code>std::vector<Item></code>	Player inventory	Player.h
<code>std::vector<Choice></code>	Story node choices	Story.h
<code>std::vector<std::string></code>	Menu options	Utils.cpp, Game.cpp
<code>std::map<int, StoryNode></code>	Story node graph (node ID → node data)	Story.h
<code>struct</code>	Item, Enemy, Choice, StoryNode	Various headers
<code>enum class</code>	CharacterClass, GameState, Color	Various headers
<code>std::string</code>	Text data, serialization	Throughout
<code>std::ostringstream / istringstream</code>	Serialization/deserialization	Player.cpp

10. KEY ALGORITHMS & LOGIC

10.1 State Machine Pattern

The game uses a finite state machine to manage game flow:

```
while (isRunning) {  
    switch (state) {  
        case GameState::MAIN_MENU:      mainMenu(); break;  
        case GameState::CHARACTER_CREATION: characterCreation(); break;  
        case GameState::PLAYING:         processStoryNode(); break;  
        case GameState::GAME_OVER:       gameOverScreen(); break;  
        case GameState::VICTORY:         isRunning = false; break;  
    }
```



```
    }
}
```

10.2 Damage Calculation

Damage = max(1, attacker_attack - defender_defense/3 + random(-2, +2))

- Mage class: 25% chance of critical hit (2x damage)
- Defend action: halves incoming damage

10.3 Leveling Formula

Experience to next level = previous_threshold × 1.5

Level up grants: +15 HP, +3 ATK, +2 DEF, full heal

10.4 Ending Determination

```
if (karma >= 10 && hasItem("Crystal of Light") && hasItem("Light Amulet"))
    → Secret Ascended Ending
else
    → Standard ending based on player choice
```

10.5 Serialization (Save/Load)

Player data is serialized to a pipe-delimited string:

name|classID|health|maxHealth|attack|defense|gold|level|exp|expNext|karma|itemCount|item1_name



11. HOW TO COMPILE AND RUN

Prerequisites

- Install MinGW (g++ compiler) from <https://www.mingw-w64.org/>
- Ensure g++ is added to the system PATH

Compilation

Open Command Prompt in the project directory and run:

```
g++ -o eldoria.exe main.cpp Game.cpp Player.cpp Story.cpp Combat.cpp Utils.cpp SaveSystem.cpp
```

Execution

eldoria.exe

12. SAMPLE GAMEPLAY

Main Menu

```
+=====+
|      THE LOST KINGDOM OF ELDORIA      |
+=====+
```

A Text-Based Adventure Game in C++
Developed for Pinnacle Labs Internship

```
+-----+
|  MAIN MENU  |
+-----+
```

[1] New Game
[2] About
[3] Quit

Character Creation

```
+-----+
|  CHARACTER CREATION  |
+-----+
```

Choose your class:

[1] WARRIOR
HP: 150 | ATK: 12 | DEF: 8
A stalwart fighter with high health and defense.

[2] MAGE
HP: 90 | ATK: 18 | DEF: 4
Powerful attacks with 25% critical hit chance.

[3] ROGUE

HP: 110 | ATK: 14 | DEF: 6 | +50 Gold
Balanced stats with bonus gold and 75% flee chance.

Combat Encounter

=====

⌘ COMBAT ENCOUNTER ⌘

=====

A Forest Wolf appears!
The wolf snarls, baring its fangs!

Forest Wolf HP: [████████████████████] 45/45
Your HP: [████████████████████] 150/150

- [1] Attack
- [2] Defend
- [3] Use Item
- [4] Flee

13. TESTING & VALIDATION

Test Case	Expected Result	Status
New game creation	Character creation screen appears	✓ Pass
Class selection (1/2/3)	Correct stats assigned per class	✓ Pass
Invalid input (letters, out-of-range)	Error message, re-prompt	✓ Pass
Combat — Attack	Damage dealt, enemy HP reduced	✓ Pass
Combat — Defend	Incoming damage halved	✓ Pass
Combat — Use Item	Item consumed, effect applied	✓ Pass
Combat — Flee (Rogue)	75% success rate	✓ Pass
Combat — Player death	Redirects to death ending	✓ Pass
Level up	Stats increase, full heal, notification	✓ Pass
Save game	savegame.dat created	✓ Pass
Load game	Player state restored correctly	✓ Pass
Karma tracking	Karma changes reflected in title	✓ Pass

Test Case	Expected Result	Status
All 5 endings reachable	Each ending displays correctly	✓ Pass
Inventory full (15 items)	"Inventory full" message shown	✓ Pass
Key items cannot be discarded	Error message displayed	✓ Pass

14. CHALLENGES FACED & SOLUTIONS

Challenge	Solution
ASCII art too wide for console	Replaced with compact text-art banner
<code>std::this_thread::sleep_for</code> not working on MinGW	Used Windows-native <code>Sleep()</code> API
Complex branching logic	Implemented a graph-based story system with <code>std::map</code>
Preventing invalid input crashes	Robust input validation with <code>cin.clear()</code> and <code>cin.ignore()</code>
Serializing complex player state	Custom pipe-delimited serialization format
Balancing combat difficulty	Tuned enemy stats and class-specific bonuses

15. FUTURE ENHANCEMENTS

- **Multiple save slots** — Allow saving to different files
- **Sound effects** — Console beep sounds for combat and events
- **Merchant system** — Buy/sell items with gold
- **Difficulty levels** — Easy, Normal, Hard with scaled enemy stats
- **Procedural dungeons** — Randomly generated side areas
- **Achievements system** — Track player accomplishments
- **Extended story** — Additional chapters and story arcs

16. CONCLUSION

"The Lost Kingdom of Eldoria" successfully demonstrates proficiency in C++ programming and object-oriented design principles. The game features a rich, branching narrative with 25+ story nodes,

5 unique endings, a strategic turn-based combat system, inventory management, a karma-based morality system, and file-based persistence.

The modular architecture ensures clean separation of concerns, making the codebase maintainable and extensible. The project fulfills all requirements of the Pinnacle Labs internship task — creating an interactive text-based adventure game with a captivating story, meaningful user choices, and variable outcomes.

Key technical highlights include:

- Finite state machine for game flow management
- Graph-based narrative system using `std::map`
- Serialization/deserialization for save game persistence
- Windows Console API integration for colored text output
- Robust input validation preventing runtime crashes

17. SOURCE CODE FILE LISTING

File	Lines	Size	Purpose
main.cpp	32	895 B	Entry point
Game.h	25	652 B	Game engine header
Game.cpp	387	13.9 KB	Game engine implementation
Player.h	72	2.0 KB	Player class header
Player.cpp	252	8.8 KB	Player class implementation
Story.h	42	1.1 KB	Story system header
Story.cpp	514	23.9 KB	Story nodes & narrative
Combat.h	30	940 B	Combat system header
Combat.cpp	184	7.7 KB	Combat implementation
Utils.h	55	1.9 KB	Utilities header
Utils.cpp	218	~9.7 KB	Utilities implementation
SaveSystem.h	15	359 B	Save system header
SaveSystem.cpp	49	1.3 KB	Save system implementation
Total	~1,875	~72.3 KB	14 source files

Report prepared for Pinnacle Labs Internship Submission May 2026